JAVA
Fundamentos e Boas práticas



Agenda

1. Organização de projeto
2. Tipos de dispositivos
3. Área de atuação
4. Documentação
5. Ferramentas de lint
6. Introdução
7. Exemplos de código
8. Compilado Vs
9. Interpretado
10. Convenção de nomes Boas práticas

Equipe

Augusto Bahia
Caio Kelvin

1. Introdução ao Java

Uma linguagem popular, orientada a objetos e multiplataforma.

O QUE É JAVA?

Linguagem de programação usada para criar aplicações robustas em diferentes ambientes.

CARACTERÍSTICAS

Orientação a objetos, tipagem estática, bibliotecas amplas e gerenciamento automático de memória.

PORTABILIDADE

Com uma JVM compatível, o mesmo bytecode pode ser executado em diferentes sistemas.

IDEIA-CHAVE

Escrever uma vez e executar em diferentes plataformas, respeitando as dependências do ambiente.

2. Exemplo básico de código

Hello World: o primeiro programa em Java.

MAIN.JAVA

```
public class Main {

    public static void main(String[] args)
{
        System.out.println("Hello
World!");
    }

}
```

- **class Main**

  Declara uma classe chamada Main.

- **main**

  É o ponto de entrada da aplicação.

- **println**

  Exibe uma mensagem no console.

3. Java: compilado ou interpretado?

O processo combina compilação e execução pela máquina virtual.

CÓDIGO-FONTE

01 • .java

→

COMPILADOR JAVAC

02 • bytecode .class

→

JVM

03 • interpreta / JIT

→

EXECUÇÃO

04 • programa em ação

JAVAC

Transforma o código-fonte em bytecode.

JVM

Executa o bytecode e pode usar JIT para otimizar trechos.

4. Convenção de nomes

Nomes consistentes tornam o código mais fácil de entender.

CLASSES / PascalCase

Cliente • ContaBancaria

MÉTODOS E VARIÁVEIS / camelCase

calcularTotal() • nomeCliente

CONSTANTES / MAIÚSCULAS

LIMITE_MAXIMO

PACOTES / minúsculas

br.com.empresa.projeto

EVITE **Nomes vagos como x, coisa ou dado1. Prefira nomes que indiquem a finalidade.**

5. Boas práticas de programação

Código de qualidade é mais simples de ler, testar e manter.

LEGIBILIDADE

Indentação, nomes claros e métodos objetivos.

SIMPLICIDADE

Evitar repetição e complexidade desnecessária.

REUTILIZAÇÃO

Separar responsabilidades e reaproveitar métodos.

TRATAMENTO DE ERROS

Validar entradas e tratar exceções esperadas.

TESTES

Verificar se o comportamento está correto.

6. Organização de projeto

Uma estrutura clara facilita o desenvolvimento em equipe.

ESTRUTURA TÍPICA

src/main/java — código da aplicação
src/test/java — testes automatizados
pom.xml — dependências e configuração Maven
README.md — instruções do projeto

FERRAMENTAS ESSENCIAIS

Maven: compila, testa e gerencia dependências.
Git: registra versões e permite trabalhar em branches.
Separar código e testes facilita a manutenção.

Organizar é deixar claro onde cada responsabilidade deve ficar.

7. Dispositivos e plataformas

Java pode atender diferentes cenários de execução.

- **COMPUTADORES**
  Aplicações desktop e ferramentas.
- **SERVIDORES**
  APIs, serviços e sistemas empresariais.
- **ANDROID**
  Aplicativos móveis com ferramentas compatíveis.
- **NUVEM / EMBARCADOS**
  Serviços cloud e dispositivos especializados.

A plataforma escolhida influencia desempenho, segurança, bibliotecas e implantação.

TRANSIÇÃO → Aplicações, documentação e ferramentas de qualidade

8. Áreas de atuação

Java aparece em diferentes tipos de solução.

- **WEB**
  Back-end de sites, APIs e serviços.
- **SISTEMAS EMPRESARIAIS**
  Sistemas internos, integrações e transações.
- **ANDROID**
  Aplicativos móveis em conjunto com o ecossistema Android.
- **BIG DATA**
  Processamento de dados com ferramentas do ecossistema Java.
- **DESKTOP**
  Aplicações com interface gráfica.

9. Documentação

Documentar explica como usar o projeto e por que decisões foram tomadas.

README.md

Apresenta objetivo, pré-requisitos, instalação e comandos para executar o projeto.

COMENTÁRIOS

Explicam decisões ou trechos que não são óbvios. Evite repetir o que o código já deixa claro.

JAVADOC

Gera documentação a partir de comentários padronizados, especialmente em classes e métodos públicos.

BENEFÍCIO

Menos dúvidas, integração mais rápida de novos desenvolvedores e manutenção facilitada.

10. Ferramentas de lint

Ferramentas automatizadas ajudam a encontrar problemas e padronizar o código.

CHECKSTYLE

Verifica convenções de estilo, formatação e padrões de nomes.

PMD

Detecta padrões problemáticos, duplicação e possíveis defeitos.

SPOTBUGS

Analisa bytecode para apontar possíveis bugs.

SONARQUBE

Reúne análises de qualidade, bugs, vulnerabilidades e manutenção.

GOOGLE JAVA FORMAT

Formata o código Java de maneira consistente automaticamente.

As ferramentas complementam — mas não substituem — revisão humana e testes.

11. Conclusão

Qualidade é resultado de pequenas práticas aplicadas continuamente.

CONVENÇÕES

Nomes consistentes e código legível.

ORGANIZAÇÃO

Estrutura, testes, documentação e controle de versão.

AUTOMAÇÃO

Lint e análise estática ajudam a detectar problemas cedo.

MENSAGEM FINAL

Um bom programa não é apenas aquele que funciona: é aquele que outras pessoas conseguem entender, testar e evoluir.

OBRIGADO!